

AudioNEXT

Lautstärkeanalyse und VGA-Visualisierung auf dem Arty A7

Felix Knoll Maximilian Hafeneder Nicolas Lonhoff Markus Remy Richard
Tuch

Team 05 – Die Audiophilen
CSA - Computer System Architektur

24. Juni 2026

Gliederung

Einleitung / Motivation

Notwendige Grundlagen

Aufgabenstellung / Zielsetzung

Lösungsansatz / Konzept

Lösung / Implementierung

Evaluierung / Ergebnisse

Diskussion

Schlusszusammenfassung

Literaturverzeichnis

Quellcodeübersicht

Aufgetretene Probleme

Einleitung / Motivation [Knoll]

- AudioNEXT misst die Lautstärke eines eingehenden Audiosignals in Echtzeit
- Eingang über eine 3,5 mm Klinke und den Pmod I2S2
- Ausgabe der Lautstärke über den Pmod VGA als Text und Balken
- Reiz, ein komplettes System von der Audioquelle bis zur Anzeige selbst aufzubauen
- Eigene Hardware und Software entwerfen und ihr Zusammenspiel verstehen
- Praxisnahe Anwendung der Inhalte aus Computer System Architektur

Notwendige Grundlagen - I2S [Hafeneder]

TODO

Notwendige Grundlagen - RMS [Knoll]

- RMS (Root Mean Square) ist der quadratische Mittelwert eines Signals
- Er beschreibt die mittlere Leistung und damit die wahrgenommene Lautstärke
- Werte quadrieren, über einen Block mitteln und die Wurzel ziehen
- Linker und rechter Kanal werden gemeinsam ausgewertet

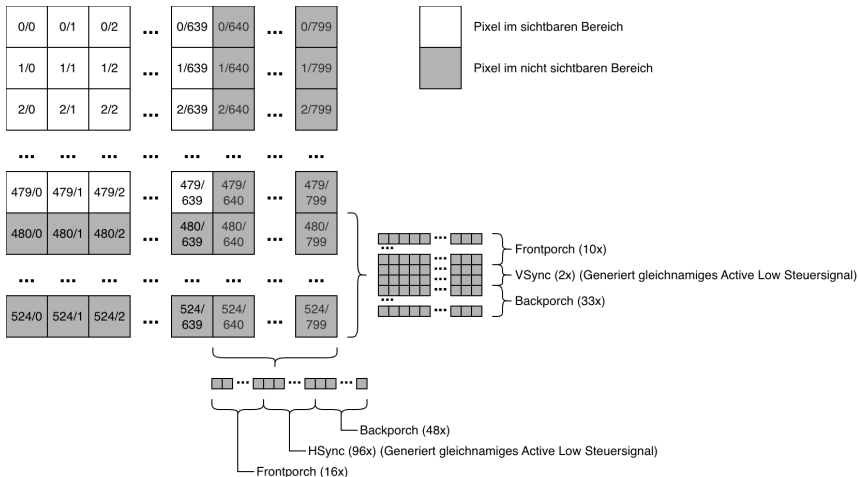
$$x_{\text{RMS}} = \sqrt{\frac{1}{2N} \sum_{i=1}^N (l_i^2 + r_i^2)}$$

- Ergebnis als Prozent der Vollaussteuerung im 7.2 Festkommaformat
- Ganzzahlige Wurzel (isqrt) für die Berechnung ohne Gleitkomma

Notwendige Grundlagen - VGA Bildaufbau [1, 2] [Remy]

- Auflösung: 640x480@60 Hz
- Signale:
 - HSync
 - VSync
 - Red
 - Green
 - Blue
- Bildaufbau (hor./vert.):
 - Aktiver Bildbereich
 - Frontporch
 - Sync
 - Backporch

Notwendige Grundlagen - VGA Bildaufbau [2] [Remy]



Aufgabenstellung / Zielsetzung [Tuch]

Aufgabe: Echtzeit-Lautstärkemessung eines Audiosignals auf dem Arty A7-100T FPGA mit visueller Ausgabe über VGA.

- Audiodaten über 3,5 mm Klinke und Pmod I2S2 empfangen (I2S, 96 kHz/Kanal)
- Zwei eigene AXI Lite Peripherals für Audio und Visualisierung entwerfen
- RMS-Lautstärke auf dem MicroBlaze in Software berechnen
- Ergebnis als Text und Balken auf einem VGA-Monitor darstellen

Abgrenzung:

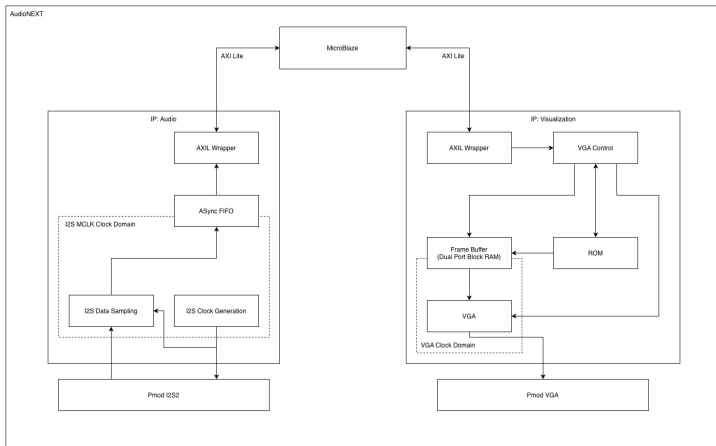
- Keine Audioausgabe oder -aufnahme, nur Pegelanalyse
- Keine Gleitkomma-Arithmetik, ausschließlich Festkomma (7.2-Format)
- Kein DMA – Datenübertragung per Register-Polling

Lösungsansatz / Konzept - [Knoll]

- Aufteilung in zwei eigene AXI IPs und den MicroBlaze als zentrale Steuerung
- Audio IP empfängt das I2S Signal, puffert die Samples und stellt sie bereit
- MicroBlaze berechnet die Lautstärke als quadratischen Mittelwert (RMS)
- Visualization IP erzeugt das VGA Bild aus einem Textframebuffer
- Zeitkritische Verarbeitung in Hardware, flexible Berechnung in Software
- Kommunikation einheitlich über AXI Lite mit Interrupts je Kanal
- Feste Registerschnittstellen ermöglichen paralleles Arbeiten im Team

Lösungsansatz / Konzept - Blockdiagramm

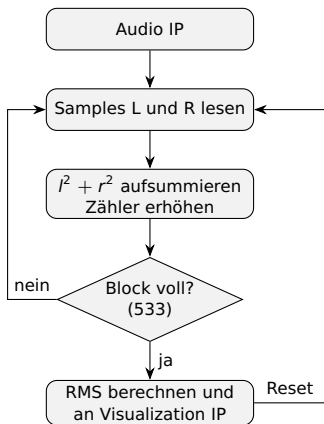
[Gesamtteam]



Lösung / Implementierung - TODO - MAX

Lösung / Implementierung - Audio Software [Knoll]

- Sampling über das CTRL Register aktivieren
- Statusbits SLA und SRA pollen
- 24 Bit Samples mit Vorzeichenerweiterung lesen
- Pufferüberläufe über DOL und DOR erkennen
- RMS berechnen und an die Visualization IP geben
- Funktionstest und Selbsttests prüfen
Register, Sampling und RMS
Berechnung automatisch beim
Programmstart



Lösung / Implementierung - Audio AXI Hardware [Tuch]

- AXI Lite Slave mit 10 Registern (5-Teile-Muster aus den Tutorials)
- IPISR mit W1C Toggle: Schreiben einer 1 kippt das Bit zurück
- Interrupt-Generierung: $GIE \wedge IPIER \wedge IPISR$ pro Kanal
- Acknowledge-Puls beim Lesen von ADATLR/ADATRR löst AXI-Stream Handshake mit FIFO aus
- `sla_flag` wird für einen Takt maskiert, damit IPISR nicht sofort erneut gesetzt wird
- Selbsttest prüft Register, Sampling Enable und beide Kanäle automatisch beim Start

Offset	Register
0x00	GSCR (Reserved)
0x04	GIER (GIE)
0x08	IPIER (SLIE, SRIE)
0x0C	IPISR (SLA, SRA)
0x10	IDR (Konstante)
0x14	VERR (Konstante)
0x18	CTRL (SEN)
0x1C	STATUS (DOL, DOR, SLA, SRA)
0x20	ADATLR (24 Bit)
0x24	ADATRR (24 Bit)

Lösung / Implementierung - VIS AXI HW [Lonthoff]

- AXI4-Lite Slave Interface für Registerzugriff
- AXI4-Stream Master Interface für Datenübertragung an VGA IP

Lösung / Implementierung - VIS CORE SW [Lonthoff]

- Software-API für die VIS CORE IP
- Initialisierung der IP und Konfiguration der Register
- Polling der IP-Statusregister (FDP, WD)
- Schreiben von Zeichen in den Textframebuffer
- Berechnung der Farbe für die Lautstärkebalken

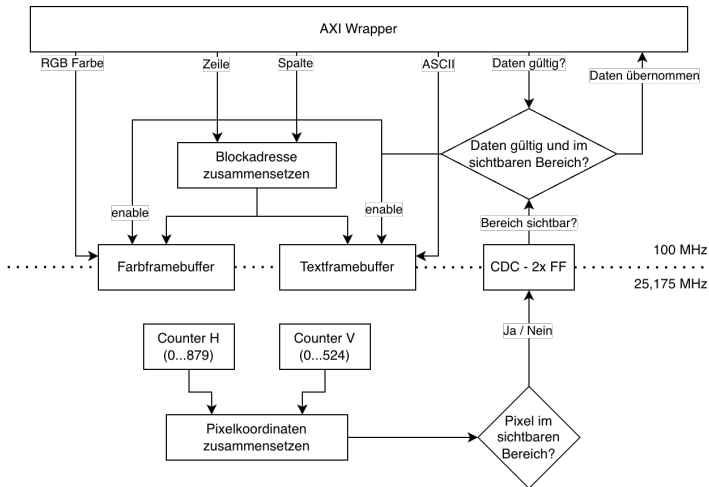
Lösung / Implementierung - VIS CORE SW TB [Lonthoff]

- Testbench für die VIS CORE Software-API
- Testet die Initialisierung, Registerkonfiguration und Zeichenanzeige
- Simuliert die Lautstärkebalkenanzeige

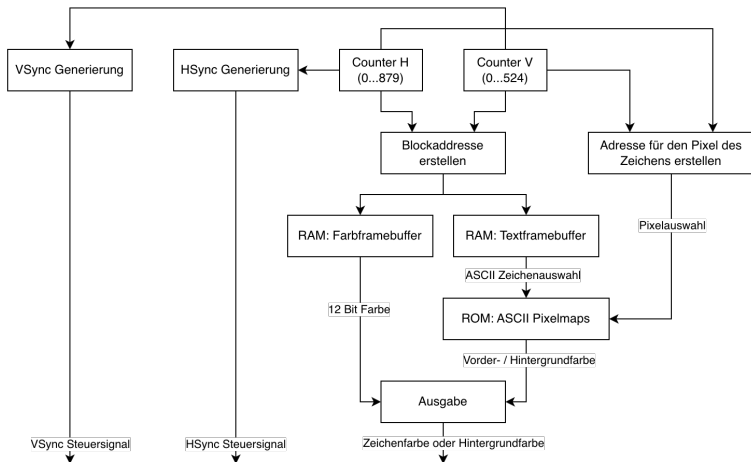
Lösung / Implementierung - VGA - Funktionsübersicht [Remy]

- Bildaufteilung in 8x16 Pixel Blöcke für je ein Zeichen
- Schreiben neuer Werte in den Farb- & Textframebuffer während des nicht sichtbaren Bereichs
- Textframebuffer: Verweis auf ASCII-Zeichen im ROM pro Block
- Farbframebuffer: 12 Bit Textfarbe pro Zeichenblock
- ROM: 8x16 Pixelmap pro Zeichen (1 = Text, 0 = Hintergrund)
- Pmod VGA zur Ausgabe des Signals

Lösung / Implementierung - VGA - Steuerung [Remy]



Lösung / Implementierung - VGA - Bildausgabe [Remy]



Lösung / Implementierung - State-Chart

@ Hafeneder?

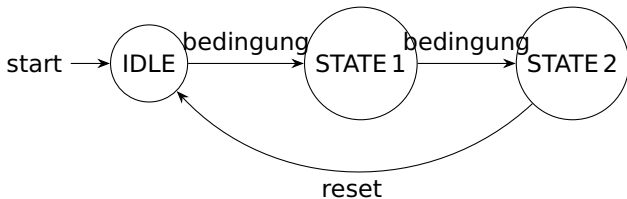


Abbildung 1: Platzhalter - State-Chart (TODO: durch reale FSM ersetzen).

Evaluierung / Ergebnisse

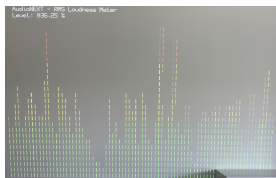
@Lonthoff

- TODO: Welche Ziele wurden erreicht? (Erfüllungsgrad)
- TODO: Messergebnisse / Tests / Demonstration.

Ziel (aus Zielsetzung)	Erfüllt	Anmerkung
TODO: Ziel 1	ja/teilw./nein	TODO
TODO: Ziel 2	ja/teilw./nein	TODO
TODO: Ziel 3	ja/teilw./nein	TODO

Evaluierung / Ergebnisse - Getestete Bildschirme [Remy]

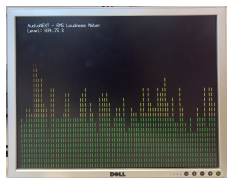
Beamer in K101



Beamer in K143



Bildschirm aus K143a



Diskussion [Tuch]

▪ Bedeutung der Ergebnisse:

- Vollständige Pipeline von der Audioquelle bis zur VGA-Anzeige funktionsfähig
- Eigene AXI IPs und Treiber zeigen das Zusammenspiel von HW und SW auf einem realen FPGA
- Selbsttests sichern die Kernfunktionen automatisch ab

▪ Ausblick:

- Interrupt-gesteuertes Sampling statt Polling für geringere CPU-Last
- Frequenzanalyse (FFT) für ein Spektrogramm auf dem VGA-Ausgang
- DMA-Anbindung für höhere Datenraten ohne MicroBlaze-Engpass

▪ Offene Punkte:

- Polling-Schleife druckt bei Timing-Schwankungen „not av“-Meldungen
- Kein Overflow-Handling in der RMS-Akkumulation bei Vollaussteuerung über lange Blöcke
- FIFO-Tiefe (CDC-Buffer) begrenzt die Toleranz gegenüber Software-Verzögerungen

Schlusszusammenfassung - [Knoll]

- AudioNEXT zeigt die aktuelle Lautstärke live auf einem VGA Monitor
- Audioerfassung, RMS Berechnung und Visualisierung greifen zuverlässig ineinander
- Eigene AXI IPs und passende Softwaretreiber wurden entwickelt
- Selbsttests und Funktionstests sichern die einzelnen Komponenten ab
- Zusammenspiel von Hardware und Software am realen FPGA System gezeigt
- Klare Modulgrenzen und Schnittstellen haben die Integration vereinfacht

Literaturverzeichnis

- [1] Digilent. *Pmod VGA Reference Manual*. URL: <https://digilent.com/reference/pmod/pmodvga/reference-manual> (besucht am 23.06.2026).
- [2] Digilent. *VGA Display Controller*. URL: <https://digilent.com/reference/learn/programmable-logic/tutorials/vga-display-controller/start> (besucht am 21.06.2026).
- [3] Digilent. *Adding DDR Memory to a Microblaze Design*. URL: <https://digilent.com/reference/programmable-logic/guides/microblaze-adding-ddr> (besucht am 21.06.2026).
- [4] Digilent. *digilent-xdc*. URL: <https://github.com/Digilent/digilent-xdc> (besucht am 21.06.2026).

Literaturverzeichnis

- [5] Daniel Münch und Sandro Amato. *VGA Demoprojekt*.
- [6] Daniel Münch. *Axi Timer Demoprojekt*.
- [7] Xilinx. *Hello World Demoprojekt*.

Quellcodeübersicht - Felix Knoll [Knoll]

- Dateien basierend auf dem Axi Timer Demoprojekt [6]:
 - aud_driver.c, aud_driver.h
 - aud_driver_g.c, aud_driver_i.h
 - aud_driver_l.c, aud_driver_l.h
 - aud_driver_sinit.c, aud_driver_sinit.h
 - aud_selftest_pio.c, aud_selftest_pio.h
- Dateien basierend auf dem Hello World Demoprojekt [7]:
 - audio_rms_main.c
 - platform.c, platform.h
- Dateien ohne Basis:
 - audio_rms.c, audio_rms.h
 - audio_rms_test.c

Quellcodeübersicht - Maximilian Hafeneder

[Hafeneder]

@Hafeneder

- Dateien basierend auf dem Axi Timer Demoprojekt:
 - aud_axislave_v1_0.vhd
 - TODO
- Dateien basierend auf der Digilent Constraints Vorlage [4]:
 - ArtyA7100.xdc
- Dateien ohne Basis:
 - TODO

Quellcodeübersicht - Nicolas Lonthoff [Lonthoff]

@Lonthoff

- Dateien basierend auf dem Axi Timer Demoprojekt:
 - vis_axislave_v1_0_S00_AXI.vhd
 - vis_axistream_v1_0.vhd

Quellcodeübersicht - Markus Remy [Remy]

- Dateien basierend auf dem VGA Demoprojekt [5]:
 - vis_core.vhd
 - vis_RAM.vhd
 - vis_vga_ctrl.vhd
 - vis_vga_pkg.vhd
- Dateien basierend auf dem Axi Timer Demoprojekt [6]:
 - vis_tb.vs
 - vip_db.bd
 - aud_axislave_v1_0.vhd
- Dateien basierend auf der Digilent Constraints Vorlage [4]:
 - ArtyA7100.xdc
- Dateien basierend auf der Digilent DDR Vorlage [3]:
 - mb_blockdesign.bd

Quellcodeübersicht - Richard Tuch [Tuch]

- Dateien basierend auf dem Axi Timer Demoprojekt:
 - `aud_axislave_v1_0_S00_AXI.vhd`
 - `aud_selftest_pio.c`, `aud_selftest_pio.h`
- Dateien basierend auf dem Hello World Demoprojekt:
 - `audio_rms_main.c` (Beiträge)
- Dateien ohne Basis:
 - `implementation_plan.md`

BACKUP

Aufgetretene Probleme [Remy][Knoll]

- Constraints:
 - Doppelte Clock Constraints durch automatische von MIG des DDRs generierte und manuell angelegte eigene Constraints
- Cache:
 - Cache in HW aktiviert, aber in SW nicht automatisch aktiviert
 - > DDR Zugriff dauerte sehr lange
 - > Manuelle Aktivierung notwendig
- VGA Timing:
 - Front- und Backporch in der VGA Vorlage vertauscht
 - > Leicht verschobenes Bild
 - Timing in der VGA Vorlage nicht exakt nach den Spezifikationen
 - > Manche Bildschirme interpretieren das Bild falsch